AI NOTES SERIES

LLM Application Development

How applications are built around large language models — tokens, prompts, retrieval, and honest limits.

By Shah Fahad

AI & Technology Innovator and Educator · Buner, Khyber Pakhtunkhwa, Pakistan

November 2026

These are introductory study notes, written in simple English for teachers, students and beginners. They come from my own learning and exploration of how these applications are put together. Explanations are simplified on purpose, so the main ideas are easy to follow and easy to revise.

What an LLM Is

A **large language model**, usually shortened to **LLM**, is a computer model that works with human language. The word "large" refers to the scale of the model and of the material it was trained on.

During training, the model is shown huge amounts of text — books, articles, websites and other written material. Again and again, it tries to predict what word, or part of a word, comes next in a piece of text. When its prediction is wrong, it adjusts itself a little. Over a very large number of examples, it learns patterns in how language is used.

> **Plain definition:** A large language model is a model trained on huge amounts of text to predict and generate language. It learns patterns in words, and it uses those patterns to write, answer, summarise and translate.

Prediction is the engine

Everything an LLM writes is built step by step from prediction. It produces a little text, adds that text to what came before, and predicts the next part again. This is why its writing can read smoothly: each new piece is chosen to fit the pieces before it.

The engine inside many tools

- A **chatbot** is an application that holds a conversation. In many modern chatbots, an LLM is the engine that produces the replies.
- Writing helpers, summarising tools, translation tools and question-answering tools often use an LLM in the same way — as the language engine inside a larger application.
- The application around the model matters too. It decides what the user sees, what instructions the model follows, and what extra information the model is given.

That last point is the subject of these notes. Building an LLM application is mostly the work of shaping what goes into the model and what happens to what comes out.

The Building Blocks

Three ideas explain most of how an LLM application works: tokens, embeddings, and the context window.

Tokens — the pieces the model actually reads

An LLM does not read text letter by letter, and it does not always read whole words either. It breaks text into small chunks called **tokens**. A token can be a whole short word, part of a longer word, or a piece of punctuation. The model reads and writes tokens, not raw text.

> **Example:** The sentence "Learning is useful" might be split into pieces such as "Learn", "ing", " is" and " useful". The exact split depends on the model.

Embeddings — meaning turned into numbers

Inside the model, each token is turned into a list of numbers called an **embedding**. The numbers capture something about meaning and use. Tokens that are used in similar ways end up with numbers that are close to each other, and tokens with very different uses end up far apart.

> **Example:** The words "king" and "queen" appear in similar kinds of sentences, so their embeddings sit close together. The word "banana" is used very differently, so its embedding sits far away from both.

The context window — what the model can consider at once

The **context window** is the limited amount of text the model can consider at one time. Everything the model works with in a single reply — the instructions, the conversation so far, any passages supplied to it, and its own growing answer — must fit inside this window.

> **Example:** Think of a desk that can hold only a certain number of pages. The model can work only with the pages on the desk. Pages that do not fit are, for that reply, as if they were not there at all.

> **Remember:** If something is not inside the context window, the model cannot use it for that answer — no matter how important it is.

Prompts and System Instructions

The model does not decide on its own what kind of helper to be. The words placed into its context window guide it, and those words come from two different places.

Prompts — the words the user gives

A **prompt** is the text a user gives the model: a question, an instruction, or material to work on. A clear prompt usually gives better results than a vague one, because the model follows patterns — and a clear request creates a clear pattern to follow.

- Say what you want, in plain words — for example, a summary, a list, or an explanation for a beginner.
- Give the material the answer should be based on, when you have it.
- Say who the answer is for, so the level and tone can fit the reader.
- Ask for one task at a time when the task is large or has several parts.

System instructions — the standing orders

A **system instruction** is a standing instruction set by the builder of the application, before the user writes anything. It stays in place across the conversation and shapes how the model behaves throughout.

- It can set the role — for example, "you are a helpful assistant for a school office".
- It can set the style — for example, short answers, simple English, or a formal tone.
- It can set limits — for example, answer only from the documents provided, and say so when the answer is not in them.

> **Simple picture:** The system instruction is like the job description given to a new worker on the first day. The prompt is like the specific task given to that worker today. Both sit inside the context window, and both shape the answer.

Good application design treats both with care. Weak instructions lead to wandering answers, even when the model itself is strong.

Retrieval-Augmented Generation (RAG)

An LLM knows patterns from its training, but it does not know your school handbook, your office records, or a new document written yesterday. **Retrieval-Augmented Generation**, shortened to **RAG**, is a design that supplies the model with the right passages at the moment a question is asked.

How RAG works, as a flow of words

The question — a user asks a question in ordinary words.

Into numbers — the question is turned into an embedding, a list of numbers that captures its meaning.

The search — a vector database compares those numbers with the numbers stored for many saved passages, and finds the passages whose meaning is most similar.

Into the context — those passages are placed into the context window, together with the question and the instructions.

The answer — the model writes its answer grounded in those passages, instead of relying only on patterns from training.

Before any question is asked, the source documents are split into passages, each passage is turned into an embedding, and those embeddings are stored in the vector database, ready to be searched.

What a vector database is, in plain words

A **vector database** is a store built for embeddings. An ordinary database finds records that match exactly — the same name, the same date. A vector database finds passages whose numbers are closest, which means passages whose meaning is most similar, even when they use different words from the question.

Where RAG is used

This is how many chatbots answer from a specific handbook or record set. The model stays the same, but because the relevant pages are placed in front of it for each question, its answers can be about that organisation's own material.

What People Build With LLMs

Most LLM applications are combinations of the same few parts: a model, instructions, sometimes retrieval, and a simple screen or chat window for the user. The differences lie in what the application is for.

Assistants that answer from an organisation's own documents

A school, office or business can connect its own documents — policies, handbooks, circulars, guides — through RAG. Staff then ask questions in ordinary language and receive answers drawn from those documents, instead of searching through files by hand.

Document question-answering

A user supplies a document — a report, a contract, a long letter — and asks questions about it. The document, or the most relevant passages from it, are placed into the context window, and the model answers from that material. This works well for finding a detail quickly or checking what a document says about a particular point.

Summarisation and drafting tools

- **Summarisation** — long text is placed in the context, and the model produces a shorter version that keeps the main points.
- **Drafting** — the user gives the purpose, the key points and the tone, and the model produces a first draft of a letter, notice, lesson outline or message for a person to check and improve.
- **Rewriting** — an existing text is made simpler, more formal, or shorter, while keeping its meaning.

> **Common pattern:** In careful applications, the model produces a draft or a suggestion, and a person checks it before it is used. The model does the fast first work; human judgement stays in charge.

What makes one application better than another

- Clear system instructions that fit the real task.
- Good source documents, kept correct and up to date, when retrieval is used.
- Honest behaviour when the answer is not in the material — saying so, instead of guessing.
- Testing with real questions before other people rely on it.

Honest Limits

RAG and careful prompts improve an application a great deal, but they do not make it always right. Retrieval does not guarantee correctness, and every builder and user should understand why.

Where a grounded answer can still go wrong

- **The search can fetch the wrong passage.** Similarity of meaning is not the same as relevance. A passage can be close in wording and still not be the passage that answers the question.
- **The right passage can be misread.** The model can misunderstand a sentence, mix two passages together, or miss an important condition written in the text.
- **The model can add unsupported claims.** Even with good passages in front of it, the model may add details that are not in them, written in the same confident style as the supported parts.
- **The source itself can be wrong or out of date.** An answer grounded in an old or mistaken document will faithfully repeat the mistake.
- **Some questions have no answer in the material.** A well-built application says so plainly. A poorly built one may invent an answer instead.

> **Grounding helps, evaluation remains essential.** Grounding — tying the answer to supplied passages — makes errors less likely and easier to check. It does not remove the need to test the application, read its answers critically, and keep a person responsible for important decisions.

Good habits for users

- Check important answers against the original document, not only against the chatbot's reply.
- Be more careful when the cost of an error is high — money, health, law, or someone's official record.
- Treat a confident tone as a style of writing, not as proof that an answer is correct.

Words to Know

LLM — a large language model: a model trained on huge amounts of text to predict and generate language.

Token — a small chunk of text, such as a word, part of a word, or a punctuation mark, which is the unit the model actually reads and writes.

Embedding — a list of numbers that represents a token or passage and captures something about its meaning, so that similar meanings have similar numbers.

Context window — the limited amount of text the model can consider at one time; everything it uses for an answer must fit inside it.

Prompt — the text a user gives the model: a question, an instruction, or material to work on.

System instruction — a standing instruction set by the builder of an application that shapes how the model behaves throughout a conversation.

RAG — Retrieval-Augmented Generation: a design in which relevant passages are searched for and placed into the context, so the model answers grounded in them.

Vector database — a store for embeddings that finds passages whose meaning is most similar to a question, rather than only exact matches.

Grounding — tying an answer to supplied source material, so the answer is based on that material instead of only on patterns from training.

Chatbot — an application that holds a conversation with a user; in many modern chatbots, an LLM produces the replies.

Quick Revision

- An LLM is trained on huge amounts of text to predict and generate language. It is the engine inside chatbots and many AI tools.
- The model reads tokens — small chunks of text — not whole pages at once.
- An embedding turns a token or passage into numbers that capture meaning; similar meanings sit close together.
- The context window is limited. If something is not inside it, the model cannot use it for that answer.
- A prompt is what the user gives; a system instruction is the standing instruction set by the builder. Both shape the answer.
- In RAG, the question is turned into numbers, a vector database finds the most similar stored passages, those passages enter the context, and the model answers grounded in them.
- People build document assistants, document question-answering, and summarisation and drafting tools from these parts.
- Retrieval does not guarantee correctness: the wrong passage can be fetched, the right passage can be misread, and unsupported claims can still be added.
- Grounding helps, and evaluation remains essential. A person stays responsible for important answers.

Closing Note

These notes are an introduction to LLM application development. Explanations are simplified on purpose, so a beginner can build a clear picture first and add detail later. They are study notes from my own learning and exploration of how these applications are put together, shared for students, teachers and anyone starting out with this subject.

Prepared by Shah Fahad